SE Lab 3 – Architecture Justification

Topic: Hostel Management
Team No: 07
Section: B

Architecture Selection

We chose Client-Server Architecture for the Hostel Maintenance & Issue Ticketing System. Residents use the Resident App and the warden uses the Warden Dashboard. Both are clients that connect over HTTPS to one central server. The server contains the Ticket Manager, Dispatch Service and SLA Escalation Monitor, and it uses the Ticket Database and an SMS/Email Notification Gateway.

Reason 1

Many residents log tickets from their own phones, and the warden must see all of them in one queue. If every ticket is stored on one central server, the resident, the warden and the maintenance staff always see the same ticket ID, status and timeline. This also makes it easy to keep the data consistent, which is a main advantage of Client-Server.

Reason 2

The 24-hour SLA escalation (NFR-001) has to run all the time, even when no resident or warden has the app open. In a Client-Server design the SLA Escalation Monitor runs on the server, checks open tickets regularly and sends the SMS/email alert through the Notification Gateway. Microservices would be too complex for a single hostel's ticketing system and would add network calls and deployment work without any real need to scale parts separately.

Security Advantage

All data stays on the central server and is reached only through the server's APIs, so login and role checks happen in one place. A resident can only view and close their own tickets, and only the warden can assign or reassign staff. Room numbers, photos and phone numbers are stored in the Ticket Database on the server, not on residents' phones, and all communication with the clients uses HTTPS.

Performance Benefit

The clients are thin. They only send a ticket or request a list, and the server does the work. Ticket Manager, Dispatch Service and SLA Monitor run on the same server, so calls between them are fast internal calls instead of network requests. The SLA check is a single indexed database query on open tickets, so overdue high-priority tickets are found quickly and the alert goes out on time.

Trade-off

The central server is a single point of failure. For one hostel this is acceptable, and the risk can be reduced with regular database backups and a standby server.